

Unidad 2: Distintos modelos NoSQL

Clase 6 – Clave–valor y procesamiento de alta performance

Objetivo

Analizar el modelo clave–valor y aplicar estructuras básicas de Redis para resolver escenarios de alto rendimiento, gestión de estado y reglas temporales en procesos automatizados de análisis y control.

Tema

- Modelo Clave / Valor.
- Redis (commands)
 - Claves, hashes, sets, contadores
 - TTL y ventanas temporales
 - Caché, reglas rápidas y estado
- Casos de uso en auditoría y control automatizado.

Por qué existe el modelo clave–valor

Cuando se estudian bases de datos, suele aparecer primero la idea de almacenar información rica en relaciones, tablas o documentos. Sin embargo, en muchos sistemas modernos ese no es siempre el problema principal. Hay escenarios en los que lo más importante no es representar estructuras complejas, sino responder en milisegundos, mantener un estado momentáneo o recuperar un dato puntual de manera inmediata. Allí aparece el sentido del modelo clave–valor.

Una forma simple de empezar a comprenderlo es con esta pregunta: ¿qué ocurre cuando no necesito recorrer muchas relaciones, ni analizar documentos extensos, sino simplemente guardar algo con un identificador y recuperarlo lo más rápido posible? En esos casos, el modelo clave–valor ofrece una solución extremadamente eficiente. Su fortaleza no está en la complejidad de la consulta, sino en la velocidad y simplicidad del acceso.

“Si en MongoDB pensamos en documentos, en Redis pensamos en acceso inmediato por identificador.”

Mientras en una base documental se guarda una estructura más rica y flexible, en un modelo clave–valor se privilegia la identificación rápida de un dato mediante una clave única.

¿Qué es una clave y qué es un valor?

En este modelo, la información se organiza en pares. Cada par está formado por una clave y un valor.

La clave es el identificador. Es el nombre o referencia que permite ubicar el dato. Debe ser única dentro del espacio donde se almacena, porque si dos elementos tuvieran la misma clave, uno sobrescribiría al otro o generaría conflicto.

El valor es el contenido asociado a esa clave. Puede ser un texto, un número, un estado, una estructura pequeña o incluso un conjunto de datos más complejo, según la tecnología utilizada.

Si lo llevamos a una analogía cotidiana, la clave sería como la etiqueta pegada en un cajón, y el valor sería lo que está guardado dentro. Si el cajón dice “usuario:1001”, lo importante es que, al leer esa etiqueta, el sistema vaya directamente a su contenido sin tener que revisar todos los cajones del depósito.

Por ejemplo:

- clave: sesion:usuario_45
- valor: activa

O bien:

- clave: contador:intentos:cliente_302
- valor: 4

En ambos casos, la lógica es la misma: existe una referencia clara, y a partir de ella se accede al dato de forma inmediata.

La gran ventaja de este modelo es que el acceso no depende de recorrer una estructura compleja. No se parte de una tabla para filtrar muchas filas, ni de un documento para buscar un atributo entre cientos de campos. Se parte directamente de la clave.

Esto significa que el sistema no necesita “pensar demasiado” para encontrar la información. Si conoce la clave, puede ir casi de manera directa al valor asociado. Por eso se dice que este modelo tiene una lógica de acceso inmediato.

Didácticamente, conviene imaginar dos situaciones distintas. En la primera, se tiene una biblioteca sin catálogo digital y hay que buscar un libro recorriendo estantes, leyendo autores, títulos y temas. En la segunda, se tiene un casillero numerado y el dato exacto de qué casillero abrir. El modelo clave–valor funciona como este segundo caso: el acceso es directo porque el punto de entrada ya está definido.

Esta característica lo vuelve especialmente útil cuando un sistema necesita resolver millones de lecturas o escrituras pequeñas en muy poco tiempo. En lugar de invertir recursos en interpretar una consulta compleja, responde casi instantáneamente porque ya sabe dónde mirar.

Buscar por clave exacta no es lo mismo que hacer consultas complejas

Este punto es clave para evitar una confusión frecuente. El modelo clave–valor no está pensado para consultar información con la misma flexibilidad que un sistema relacional o documental.

Si en una base relacional se quisiera buscar “todos los clientes de una provincia con más de tres compras durante el último mes”, el motor de base de datos podría usar filtros, comparaciones, agrupamientos y joins. En una base documental también podría hacerse una búsqueda por distintos campos y condiciones. Es decir, esos modelos permiten explorar los datos con más profundidad.

En cambio, en el modelo clave–valor la pregunta típica no es “¿qué registros cumplen determinada condición?”, sino “dame el valor asociado a esta clave exacta”.

Esa diferencia cambia por completo la lógica del diseño. En clave–valor no se modela pensando en análisis abiertos, sino en accesos concretos y repetitivos. Si el sistema necesita saber rápidamente si un usuario está bloqueado, cuánto vale un contador o cuál es el estado actual de un proceso, este modelo resulta ideal. Pero si se quiere realizar una exploración compleja o un análisis multidimensional, probablemente no sea la mejor opción por sí solo.

Por eso, cuando se diseña con esta lógica, una buena práctica actual es preguntarse primero qué accesos exactos y repetitivos necesita resolver el sistema. El modelo clave–valor funciona mejor cuando el patrón de consulta está muy claro desde el inicio.

Por qué es útil en sistemas de alto rendimiento

Hoy muchas aplicaciones digitales trabajan bajo exigencias muy distintas a las de hace algunos años. No solo deben almacenar información; también deben responder rápido, escalar, soportar miles o millones de interacciones y mantener estados operativos en tiempo real. En esos contextos, el rendimiento deja de ser un detalle técnico y se convierte en un requisito central del negocio.

El modelo clave–valor es útil en estos escenarios porque reduce el costo de acceso a la información. Si una plataforma necesita validar sesiones activas, controlar intentos fallidos, mantener un contador de eventos, consultar el estado de una tarea automática o recuperar rápidamente una regla temporal, este enfoque permite hacerlo con gran eficiencia.

Pensemos en algunos ejemplos actuales:

Una plataforma de comercio electrónico puede necesitar conocer en el instante si un producto está reservado por unos segundos mientras otro usuario avanza en el checkout. No se trata de consultar un historial completo, sino de saber un estado puntual ya mismo.

Un sistema de autenticación puede requerir contar cuántos intentos fallidos tuvo un usuario en los últimos minutos. Allí importa la velocidad, porque la decisión de bloquear o permitir acceso debe tomarse de inmediato.

En un proceso automatizado de auditoría, puede ser necesario saber si una validación ya fue ejecutada, cuál fue su último resultado o si una excepción sigue vigente. Es información de estado, operativa y transitoria, donde la rapidez vale más que la riqueza analítica.

Desde la arquitectura moderna de sistemas, este tipo de soluciones suele utilizarse como complemento de otras bases. Una base relacional puede seguir siendo el sistema de registro formal, mientras una base clave–valor resuelve el acceso inmediato y la gestión operativa del momento. Esta separación es una buena práctica vigente, porque evita forzar a una sola tecnología a resolver problemas de naturaleza muy distinta.

La idea de dato transitorio

Otro concepto fundamental del modelo clave–valor es que no siempre los datos deben vivir para siempre. Muchas veces el dato tiene valor solo durante un lapso corto, mientras un proceso está ocurriendo.

A esto se lo puede llamar dato transitorio, temporal o de vida corta. Es un dato útil para una decisión inmediata, pero que quizás pierda sentido pocas horas después o incluso unos segundos más tarde.

Por ejemplo, el estado “usuario autenticado” puede ser relevante ahora, pero dejar de serlo cuando la sesión termina. El contador de intentos fallidos puede tener valor solo mientras se analiza una posible anomalía. Una alerta momentánea puede necesitar existir durante quince minutos y luego desaparecer. En estos casos, no tiene sentido tratar toda esa información como si fuera un registro histórico permanente.

Esta mirada es muy importante en la ingeniería de datos actual. Diseñar bien no significa guardar absolutamente todo en cualquier parte, sino entender qué información necesita persistencia histórica y qué información necesita velocidad, temporalidad y caducidad.

El modelo clave–valor encaja muy bien en esa segunda necesidad. No reemplaza a un data warehouse, ni a una base relacional, ni a una documental. Cumple otra función: sostener el presente operativo del sistema.

Estado momentáneo y respuesta inmediata

Cuando se habla de estado momentáneo, se hace referencia a una condición actual del sistema que puede cambiar rápidamente.

Por ejemplo:

- una tarea está “en ejecución”

- un usuario está “bloqueado”
- una alerta está “activa”
- un proceso quedó “pendiente”
- un contador llegó a “7 eventos”

Todas esas situaciones describen algo que está ocurriendo ahora. No necesariamente representan un historial completo ni un análisis de largo plazo. Lo que se necesita es conocer el presente para tomar una decisión rápida.

Esto conecta al modelo clave–valor con muchos escenarios actuales de automatización, monitoreo y control. En lugar de consultar una gran base histórica cada vez que sucede un evento, el sistema mantiene estados ágiles que permiten actuar con baja latencia. Desde el punto de vista del negocio, esto mejora tiempos de respuesta, reduce carga innecesaria y permite reaccionar frente a eventos críticos con mayor precisión.

En auditoría automatizada, por ejemplo, puede ser necesario detectar si una operación ya fue observada, si una regla se activó hace pocos segundos o si un proceso quedó detenido en una etapa específica. En todos esos casos, la información importante no es una gran historia, sino el estado presente.

Un cambio de mentalidad en el diseño

El modelo clave–valor obliga a pensar distinto. No se diseña partiendo de entidades complejas y relaciones extensas, sino de necesidades concretas de acceso rápido.

La pregunta de diseño deja de ser “¿cómo represento todo el universo del negocio?” y pasa a ser “¿qué dato puntual necesito recuperar o actualizar de manera inmediata?”.

Ese cambio parece pequeño, pero es muy profundo. En tecnologías como Redis, por ejemplo, una buena definición de claves es una práctica esencial. Las claves deben tener sentido, ser consistentes, mantener una convención clara y anticipar los accesos que luego hará la aplicación. En otras palabras, en este modelo el diseño no gira tanto alrededor de tablas o documentos, sino alrededor de patrones de acceso.

Por eso, en las buenas prácticas actuales, se recomienda construir nombres de clave comprensibles, escalables y coherentes. Una clave como `control:fraude:cliente_245` comunica mucho mejor su propósito que una etiqueta ambigua o desordenada. Diseñar bien una clave es diseñar bien el acceso.

Supongamos que un sistema de control automatizado necesita saber si un proveedor está bajo observación temporal. En un modelo clave–valor podría existir una clave como `proveedor:203:observado` y un valor como `si`. La pregunta del sistema sería directa: ¿existe esa clave o qué valor tiene? No necesita consultar un universo completo de proveedores.

Pensemos ahora en un contador de intentos fallidos de login. La clave podría ser `login:intentos:usuario_18` y el valor 3. Cada nuevo intento actualiza el valor. Nuevamente, la lógica es simple: identificar y actuar.

Otro ejemplo muy actual es el de una sesión de usuario. La clave `sesion:abc123` puede representar una sesión activa y el valor almacenar un identificador, un estado o alguna información mínima asociada. El sistema no necesita leer una gran estructura compleja; le basta con saber si esa sesión existe y qué estado tiene.

Estos ejemplos muestran algo importante: el modelo clave-valor no pretende describir toda la realidad del negocio. Su objetivo es resolver con velocidad determinadas necesidades operativas.

El modelo clave-valor existe porque en muchos sistemas modernos la prioridad no es consultar relaciones complejas, sino acceder rápido a datos concretos. Su unidad básica es el par clave-valor: una clave única identifica un contenido asociado. Su fortaleza está en el acceso directo, no en la exploración abierta de los datos. Por eso resulta especialmente útil en escenarios de alto rendimiento, gestión de estado, datos transitorios, caché, contadores y reglas inmediatas.

Entender esto permite ubicar correctamente a tecnologías como Redis dentro del ecosistema NoSQL. No se trata de preguntarse si son “mejores” o “peores” que una base relacional o documental, sino de reconocer que resuelven un problema distinto.

El modelo clave-valor está pensado para responder rápido ante una clave conocida, manteniendo estados o datos operativos que muchas veces son temporales y requieren acceso inmediato.

Qué problemas resuelve Redis

Redis no debe pensarse solo como “otra base NoSQL”, sino como una tecnología concreta pensada para resolver problemas donde la velocidad de acceso, el manejo de estado y las operaciones simples pero frecuentes son más importantes que la complejidad del modelado. La documentación oficial lo presenta como un almacén de datos en memoria con persistencia en disco disponible, y además remarca que soporta distintos tipos de datos con operaciones atómicas sobre ellos. Eso ya da una pista importante: Redis no está orientado únicamente a guardar información, sino también a operar sobre ella de forma muy eficiente.

Trabajar “en memoria” significa que los datos activos residen en RAM y no dependen, en cada lectura o escritura, de ir a buscarlos a disco. Por eso Redis consigue velocidades muy altas de lectura y escritura frente a soluciones pensadas principalmente para almacenamiento persistente en disco. La propia documentación de Redis explica que esta arquitectura logra gran velocidad porque opera en memoria

y porque la representación interna de las estructuras de datos resulta más simple de manipular que en un sistema puramente orientado a disco.

Ahora bien, que Redis trabaje en memoria no significa que todo sea efímero o descartable. Redis ofrece varias estrategias de persistencia. Puede guardar snapshots puntuales del dataset mediante RDB, registrar las escrituras en un Append Only File o AOF, combinar ambos mecanismos, o incluso desactivar completamente la persistencia cuando el caso de uso es puramente de caché. RDB favorece backups compactos y reinicios más rápidos, mientras que AOF prioriza mayor durabilidad porque registra las operaciones de escritura y, con la política por defecto de sincronización cada segundo, reduce la posible pérdida a alrededor de un segundo de datos ante una caída.

Este punto es fundamental para estudiar Redis con criterio. No todos los datos del sistema tienen el mismo valor ni la misma vida útil. Algunas piezas deben conservarse como registro formal e histórico; otras solo deben existir mientras un proceso está vivo, una sesión está abierta o una regla temporal está activa. Por eso, en arquitecturas modernas, Redis suele complementar a otras bases en lugar de reemplazarlas. Una base relacional puede seguir siendo la fuente formal de verdad del negocio, mientras Redis sostiene el estado operativo del momento, acelera lecturas frecuentes o administra datos temporales que necesitan respuesta inmediata. Esta división surge naturalmente de las capacidades que la documentación oficial destaca para Redis: caché, sesiones, colas, procesamiento de eventos y estructuras rápidas en memoria.

Desde un punto de vista práctico, Redis resuelve muy bien escenarios como caché de consultas frecuentes, almacenamiento de sesiones, contadores de eventos, colas simples con listas, expiración automática de datos y reglas temporales. La documentación oficial menciona el caché como uno de sus usos más comunes, muestra que el almacenamiento de sesiones es un caso muy habitual y recuerda que las listas se usan frecuentemente para implementar pilas y colas. A esto se suman los contadores y las estructuras de pertenencia rápida, que son especialmente útiles en procesos automatizados de auditoría, monitoreo o control.

Pensemos en algunos ejemplos cercanos a la materia. Un sistema de control automatizado puede necesitar saber si una validación ya fue ejecutada, cuántas veces falló un usuario en los últimos minutos, si una alerta sigue vigente o si un proveedor está momentáneamente observado. En todos esos casos, el valor del dato está en el presente operativo, no en una consulta histórica compleja. Redis brilla justamente ahí: cuando el sistema necesita responder rápido, actualizar rápido y, muchas veces, dejar que ciertos datos expiren solos cuando ya no tienen utilidad. La propia documentación sobre sesiones, por ejemplo, destaca que Redis permite expiración automática con TTLs y lecturas y actualizaciones rápidas para pequeñas piezas de estado por usuario.

Desde las buenas prácticas actuales, la pregunta correcta no es “¿puedo guardar esto en Redis?”, sino “¿este dato necesita inmediatez, temporalidad o acceso muy frecuente?”. Si la respuesta es sí, Redis probablemente tenga sentido. Si en cambio el

requerimiento principal es trazabilidad formal, análisis histórico complejo o consistencia relacional rica, entonces Redis debería ocupar un rol complementario y no central. Esa manera de decidir evita dos errores comunes: querer usar Redis como si fuera un sistema relacional completo, o subestimarlo como si solo sirviera de caché. En realidad, es mucho más útil cuando se lo ubica en el lugar correcto dentro de la arquitectura.

Estructuras básicas de Redis

Una vez entendido para qué existe Redis, recién ahí conviene pasar a sus estructuras. Esto es importante didácticamente, porque si se empieza por los comandos aislados, el alumno puede memorizar sintaxis sin comprender el propósito. Redis organiza gran parte de su lógica alrededor de claves y tipos de datos asociados a esas claves. No solo almacena valores: ofrece estructuras nativas y operaciones específicas para cada una. Esa es una de sus fortalezas más importantes, ya que permite modelar necesidades concretas sin recurrir a capas extra de complejidad.

La lógica básica arranca con cuatro comandos muy simples: SET, GET, DEL y EXISTS. SET asigna un valor a una clave y la crea si no existe; si ya existe, la sobrescribe. GET devuelve el valor de una clave de tipo string. DEL elimina una o más claves. EXISTS permite comprobar si una clave existe. Estos comandos son el punto de partida porque permiten entender la unidad mínima de trabajo en Redis: una clave identificable y un valor asociado.

Desde una mirada de buenas prácticas, conviene que las claves tengan una convención clara y legible. En proyectos reales suele trabajarse con “namespaces” separados por dos puntos, por ejemplo auditoria:lote:2026:estado o login:intentos:usuario_45. Esto no es un requisito técnico impuesto por Redis, pero sí una práctica muy recomendable para ordenar el espacio de claves, facilitar la lectura humana y evitar ambigüedades cuando la solución crece.

Claves simples

Las claves simples son la forma más directa de uso. Se guarda un valor por clave y se lo recupera por esa misma clave. Es el modelo más cercano a un diccionario clásico. SET crea o actualiza la clave, GET lee el valor, EXISTS verifica presencia y DEL elimina la entrada. Además, SET admite opciones de expiración y de escritura condicional, lo que lo vuelve más versátil de lo que aparenta en un primer contacto.

Por ejemplo, si se quisiera guardar el estado actual de una tarea automática, podría hacerse así:

```
SET tarea:reproceso_001 "en_ejecucion"
GET tarea:reproceso_001
```


EXISTS tarea:reproceso_001

DEL tarea:reproceso_001

En este caso, la clave identifica una tarea puntual y el valor refleja su estado actual. Esta lógica también sirve para guardar el último resultado de un control, una bandera de validación rápida o el estado momentáneo de un proceso.

Didácticamente, conviene entender que este uso no está pensado para “consultar” de muchas maneras diferentes, sino para recuperar un dato puntual en forma inmediata. Es excelente cuando la aplicación ya sabe exactamente qué quiere buscar. Por eso se usa mucho para estado operativo, caché o resultados transitorios.

Hashes

Cuando una misma entidad tiene varios atributos relacionados, empezar a crear muchas claves sueltas puede volverse incómodo. En esos casos aparecen los hashes. La documentación oficial los describe como estructuras de tipo registro compuestas por pares campo–valor, muy útiles para representar objetos básicos o agrupar contadores y atributos. En otras palabras, si una clave simple guarda una sola pieza de información, un hash permite guardar varias bajo una misma clave principal.

Supongamos un proceso de auditoría del que queremos almacenar estado, fecha, usuario y resultado. En lugar de crear cuatro claves separadas, puede usarse un hash:

HSET auditoria:proc_101 estado "finalizado" fecha "2026-04-12" usuario "aflores" resultado "sin_desvios"

HGET auditoria:proc_101 estado

HGETALL auditoria:proc_101

Aquí la clave principal es auditoria:proc_101, y dentro de ella se guardan varios campos. Esto mejora la organización lógica del dato y suele facilitar el mantenimiento cuando se trabaja con entidades pequeñas o medianas.

Una buena práctica actual es usar hashes cuando varios atributos pertenecen claramente a un mismo objeto operativo. En cambio, si cada dato va a tener ciclos de vida muy distintos o se accederá de forma totalmente independiente, puede seguir teniendo sentido usar claves separadas. También conviene tener criterio con HGETALL: la documentación indica que devuelve todos los campos y valores del hash y que su costo crece con el tamaño de la estructura, por lo que es una opción razonable para hashes acotados, no para traer indiscriminadamente estructuras enormes.

Sets

Los sets son conjuntos de elementos únicos, sin repetidos. Esta estructura es muy útil cuando el problema no consiste en guardar atributos detallados, sino en saber si un elemento pertenece o no a un grupo. SADD agrega miembros a un set y, si el miembro ya estaba, simplemente no lo duplica. SISMEMBER permite verificar pertenencia.

SINTER calcula la intersección entre sets, es decir, los elementos comunes a varios conjuntos.

Pensemos en algunos casos típicos de auditoría y control. Se puede tener un set de usuarios bloqueados, otro de proveedores observados y otro de transacciones marcadas para revisión. La lógica sería muy natural:

```
SADD usuarios:bloqueados "u102" "u330"  
SISMEMBER usuarios:bloqueados "u102"  
SADD proveedores:observados "prov_22" "prov_45"  
SADD transacciones:revision "trx_9001" "trx_9002"
```

La gran ventaja de esta estructura es que evita duplicados y resuelve muy bien preguntas del tipo “¿este elemento ya está marcado?”. Esa pregunta aparece constantemente en controles automáticos.

Además, los sets permiten comparaciones entre grupos. Por ejemplo, si se tuviera un conjunto de proveedores observados y otro conjunto de proveedores con pagos recientes, una intersección puede mostrar cuáles están en ambos grupos:

```
SINTER proveedores:observados proveedores:con_pagos_recientes
```

Ese tipo de operación resulta muy valioso para microanálisis rápidos y reglas automáticas.

Como buena práctica, los sets convienen cuando importa la unicidad y la pertenencia, no el orden ni atributos detallados. También es útil recordar que comandos como SMEMBERS devuelven todo el contenido del set y la documentación los marca como operaciones más pesadas que una simple verificación de pertenencia; por eso, en conjuntos grandes, suele ser preferible consultar membresía puntual con SISMEMBER antes que traer el grupo completo sin necesidad.

Contadores

Uno de los usos más potentes y didácticos de Redis aparece con los contadores. INCR incrementa el valor entero de una clave en uno, y DECR lo decrementa. Si la clave no existe, ambos comandos parten de cero antes de aplicar la operación. La documentación oficial destaca justamente esa lógica, lo que vuelve a Redis especialmente cómodo para conteos rápidos de eventos, intentos o excepciones.

Por ejemplo:

```
INCR login:intentos:usuario_45  
INCR login:intentos:usuario_45  
GET login:intentos:usuario_45  
DECR login:intentos:usuario_45
```

Este patrón sirve para contar intentos fallidos de acceso, cantidad de alertas dentro de una ventana temporal, excepciones detectadas por proceso o veces que se ejecutó una validación automática. Como Redis soporta operaciones atómicas sobre sus tipos de datos, estos contadores son muy útiles en escenarios concurrentes donde varios eventos pueden llegar casi al mismo tiempo.

Desde el punto de vista del diseño, un contador en Redis no es solamente un número: es una forma de traducir eventos del negocio a estado operacional visible y accionable. Si un usuario supera cierto umbral, se dispara una regla. Si un proceso acumula muchas excepciones, se eleva una alerta. Si un control ya fue ejecutado demasiadas veces en un período breve, se investiga el comportamiento. Es decir, Redis no solo guarda un dato, sino que ayuda a sostener decisiones automáticas basadas en ese dato.

Lo más importante de Redis no es la cantidad de comandos, sino la lógica de elección de estructura. Si necesito guardar un único estado puntual, una clave simple puede alcanzar. Si tengo varios atributos de una misma entidad, probablemente convenga un hash. Si quiero pertenencia sin repetidos, necesito un set. Si debo acumular eventos, un contador resuelve el problema con enorme sencillez.

Ese es el verdadero aprendizaje técnico de este bloque: Redis no solo almacena datos, sino que ofrece estructuras pensadas para operar sobre ellos con rapidez y con una muy buena adecuación al problema. Elegir bien la estructura no es un detalle de sintaxis; es una decisión de diseño. Y en sistemas de alto rendimiento, una buena decisión de diseño vale tanto como escribir bien el comando.

TTL y ventanas temporales

En este bloque Redis deja de verse como una simple base de datos rápida y empieza a mostrar su valor como herramienta operativa. La idea central es esta: no toda la información debe vivir para siempre. Hay datos que solo tienen sentido durante unos segundos, unos minutos o unas horas. Redis resuelve muy bien ese problema porque permite asociar a una clave un tiempo de vida, conocido como TTL, de modo que cuando ese tiempo se cumple, la clave se elimina automáticamente. La documentación oficial explica justamente que la expiración de claves permite definir un timeout y que, al finalizar, la clave desaparece del conjunto de datos.

El TTL, entonces, no es solo un detalle técnico: es una forma de modelar temporalidad. Cuando una aplicación necesita sostener un estado momentáneo, una alerta activa, una sesión o una regla válida solo por un lapso limitado, no conviene tratar ese dato como si fuera un registro histórico permanente. Redis permite trabajar con esa lógica porque una clave puede expirar en segundos o milisegundos, y además esa información de expiración se replica y se persiste en disco. Incluso si el servidor se detiene, Redis conserva la fecha de expiración y el tiempo sigue “corriendo” de forma virtual.

Desde el punto de vista práctico, el comando que suele introducir esta idea es EXPIRE, que asigna el tiempo de vida de una clave, mientras que TTL permite consultar cuánto tiempo le queda. La documentación oficial indica que TTL devuelve el tiempo restante en segundos y que puede devolver -1 cuando la clave existe pero no tiene expiración, o -2 cuando directamente ya no existe. Esto es muy útil en monitoreo o debugging, porque no solo importa saber si una clave existe, sino también si está a punto de desaparecer.

Otra idea importante para comprender bien Redis es que la expiración no depende solo de que alguien consulte la clave. Redis maneja expiración de dos maneras: una pasiva, cuando el cliente intenta acceder a una clave ya vencida, y otra activa, cuando el propio motor revisa periódicamente claves con vencimiento y elimina las que ya expiraron. Esto evita que queden “basuras” temporales dando vueltas indefinidamente y hace que la expiración sea realmente útil para automatización y control.

Un ejemplo muy claro es el bloqueo temporal de un usuario después de varios intentos fallidos. Supongamos que queremos contar intentos durante diez minutos. Podemos incrementar una clave por usuario y hacer que expire al cabo de ese período. Si supera un umbral, se crea otra clave de bloqueo también con vencimiento. El razonamiento sería este:

```
INCR login:intentos:usuario_45  
EXPIRE login:intentos:usuario_45 600
```

```
SET login:bloqueado:usuario_45 "1" EX 600  
TTL login:bloqueado:usuario_45
```

Didácticamente, lo valioso no es memorizar la sintaxis, sino entender la lógica: el dato existe mientras la regla lo necesita. Después desaparece solo, sin limpieza manual. Redis muestra un patrón oficial de rate limiting basado justamente en INCR más EXPIRE, y además advierte que, si necesitás garantizar que ambas operaciones ocurran juntas sin riesgo de inconsistencias, conviene usar MULTI/EXEC o scripting Lua para evitar condiciones de carrera.

Eso nos lleva a una buena práctica muy actual: cuando una regla temporal se apoya en varias operaciones que deben ejecutarse como una sola unidad lógica, no alcanza con “hacerlas una detrás de otra” y confiar en que todo saldrá bien. En escenarios concurrentes o distribuidos, Redis recomienda soluciones atómicas, porque dos clientes podrían leer, incrementar o expirar al mismo tiempo y dejar estados inconsistentes. Este criterio es especialmente importante en control automatizado, fraude, límites de uso y monitoreo en tiempo real.

Otro caso sencillo es mantener una alerta activa por una hora. Si una validación crítica detecta una excepción, la aplicación puede grabar una clave con expiración y usarla mientras la alerta esté vigente:

```
SET alerta:proveedor_22 "activa" EX 3600
```

Durante esa hora, cualquier componente del sistema puede consultar si la alerta sigue viva. Cuando el plazo vence, la propia clave desaparece. Ese comportamiento reduce trabajo de mantenimiento y evita que queden alertas viejas activadas por error. La lógica es la misma que la documentación describe para expiración automática de claves con TTL.

También se pueden construir ventanas temporales para contar eventos sospechosos. Por ejemplo, si se quiere saber cuántas operaciones inusuales ocurrieron en los últimos quince minutos, se puede usar un contador por intervalo y hacer que ese contador expire un poco después del cierre de la ventana. El patrón oficial de Redis para limitar operaciones por tiempo usa exactamente esta idea: un contador asociado a una clave temporal que luego se elimina sola.

El cuarto ejemplo clásico es la invalidación de caché. Muchas veces una consulta compleja contra una base principal tarda demasiado o consume recursos innecesarios si se repite una y otra vez. En ese caso, se guarda el resultado en Redis y se define un TTL razonable. Así, durante ese lapso la aplicación responde con el dato ya calculado, y luego lo vuelve a recomponer cuando vence. Redis remarca en su material de caché que esta técnica permite dar rendimiento submilisegundo a datos almacenados en bases más lentas.

Lo más importante para estudiar es que el TTL no es solo “borrar cosas después de un tiempo”. Es una herramienta de modelado operativo. Permite representar duración, caducidad, validez temporal y estado vivo del sistema. Cuando entendés esto, Redis deja de parecer una base más y empieza a verse como un motor muy adecuado para procesos que reaccionan, controlan y limpian automáticamente según el tiempo.

Caché, reglas rápidas y gestión de estado

Una vez entendidas las estructuras y el TTL, conviene agrupar los usos de Redis en tres grandes finalidades. Esta clasificación ayuda mucho al alumno porque ordena mentalmente para qué sirve Redis en la práctica. No se trata de pensar en comandos sueltos, sino en problemas recurrentes que aparecen en sistemas reales: acelerar accesos, tomar decisiones inmediatas y sostener estados transitorios. Redis, de hecho, se presenta en su documentación como una plataforma que puede usarse como base de datos, caché, motor de streaming, broker de mensajes y más.

La primera gran finalidad es el caché. Cachear significa guardar temporalmente resultados costosos de obtener para no recalcularlos o no volver a consultar una base principal una y otra vez. Esto no reemplaza la fuente formal de datos; simplemente coloca una capa rápida delante de ella. Redis destaca que, con caché, datos almacenados en bases más lentas pueden alcanzar rendimiento submilisegundo. En arquitectura moderna, esto mejora latencia, reduce carga sobre el sistema principal y hace más estable la experiencia de usuario o de consumo analítico.

La segunda finalidad son las reglas rápidas. Acá Redis no actúa tanto como almacén histórico, sino como motor de decisión inmediata. Una regla rápida es una condición simple que debe evaluarse en tiempo real: si el usuario superó un umbral, se bloquea; si una alerta existe, se detiene el proceso; si una transacción ya está marcada, se deriva a revisión. En este tipo de problemas Redis funciona muy bien porque combina operaciones en memoria, estructuras nativas y expiración automática. La documentación oficial del caso de rate limiting lo usa precisamente para este tipo de control distribuido, apoyándose en contadores, estado compartido y operaciones atómicas.

La tercera finalidad es la gestión de estado. Muchos procesos automatizados necesitan saber en qué etapa están: pendiente, en curso, finalizado, con error, reintentando, esperando validación. Esa información no siempre pertenece a un modelo histórico complejo; muchas veces es puro estado operativo del momento. Redis lo resuelve muy bien porque puede mantener pequeñas piezas de estado con lecturas y actualizaciones rápidas. La documentación oficial sobre sesiones lo muestra con claridad: una sesión puede almacenarse como un hash con campos livianos, TTL y actualizaciones frecuentes, manteniendo separada la identidad del navegador respecto del estado real guardado en el servidor.

En términos didácticos, se puede pensar así: cuando Redis se usa como caché, acelera; cuando se usa para reglas rápidas, decide; cuando se usa para gestión de estado, acompaña el flujo de un proceso. Esa mirada es especialmente útil en auditoría, monitoreo y control automatizado, donde muchas veces conviven las tres cosas al mismo tiempo. Un control puede usar caché para no recalcular, reglas rápidas para disparar excepciones y estado para registrar en qué etapa quedó el robot o el lote procesado.

Casos de uso en auditoría y control automatizado

Redis se entiende mucho mejor cuando se lo ve actuar sobre situaciones concretas. El primer caso típico es el control de intentos fallidos. Se puede usar una clave por usuario, un contador con INCR, una expiración para que la cuenta se reinicie sola y una regla que, al superar cierto umbral, cree una clave de bloqueo temporal. Este patrón no es teórico: Redis lo muestra en su documentación como una forma clásica de rate limiting y explica que, para mayor seguridad en concurrencia, la lógica puede encapsularse con transacciones o Lua.

El segundo caso es el estado de ejecución de un robot o proceso automatizado. Acá un hash encaja muy bien porque una misma entidad necesita varios atributos: estado, fecha de inicio, última etapa ejecutada, resultado de la última validación, cantidad de reintentos. Redis describe los hashes como colecciones de pares campo-valor adecuadas para representar objetos básicos y agrupaciones de contadores. Eso permite que un robot tenga una sola clave principal y dentro de ella los campos operativos que describen su situación actual.

Un tercer caso muy útil es la lista de proveedores observados. Si el objetivo es saber rápidamente si un CUIT o ID pertenece a un conjunto bajo observación, los sets son una estructura ideal. Redis define los sets como colecciones desordenadas de cadenas únicas y remarca que agregar, quitar y verificar existencia se hace en tiempo $O(1)$. En un proceso de auditoría continua, esto permite resolver preguntas del tipo “¿este proveedor ya está marcado?” con enorme rapidez.

El cuarto caso es el caché de resultados de control. Imaginá una consulta compleja que cruza datos históricos, reglas y excepciones para construir una señal de riesgo. Si esa consulta se usa varias veces en poco tiempo, no conviene recalcularla constantemente. Redis puede guardar el resultado con un TTL corto y responder al instante mientras el valor siga vigente. Esto mejora dashboards, pruebas repetitivas y procesos de validación que necesitan reusar el mismo dato durante un lapso acotado. La propuesta de caché en Redis se apoya justamente en esa lógica de acelerar accesos frecuentes a datos que viven originalmente en sistemas más lentos.

El quinto caso es la detección de frecuencia inusual. Acá el objetivo no es solo guardar un valor, sino detectar si algo ocurre demasiadas veces en un período breve. Se puede usar un contador por evento y una expiración que represente la ventana de observación. Si el contador supera el umbral antes de vencerse, se dispara una alerta. Redis usa el mismo principio en su ejemplo de limitación de llamadas: contar dentro de una ventana, dejar expirar el contador y reiniciar automáticamente el ciclo.

En todos estos escenarios aparece una buena práctica muy importante: en producción no conviene recorrer el espacio de claves con KEYS para “ver qué hay”, porque la documentación oficial advierte que puede perjudicar el rendimiento en bases grandes. Para búsquedas parciales o exploración más segura, Redis recomienda SCAN o, mejor todavía, modelar explícitamente conjuntos o estructuras auxiliares que permitan saber qué elementos pertenecen a cada grupo. En términos didácticos, esto enseña una lección de diseño: no hay que depender de búsquedas improvisadas si el modelo puede anticipar mejor los accesos.

Comparación con otros modelos NoSQL

Para cerrar con criterio, conviene comparar Redis con otros modelos NoSQL sin caer en la idea de que uno “gana” sobre el otro. Cada uno resuelve problemas distintos. MongoDB, por ejemplo, almacena registros como documentos BSON y trabaja con un modelo flexible donde los documentos de una misma colección no están obligados a tener exactamente los mismos campos. Además, su propia documentación remarca como principio de modelado que los datos a los que se accede juntos deberían guardarse juntos. Eso lo vuelve muy adecuado para representar entidades ricas y estructuras documentales más completas.

Neo4j, en cambio, modela datos como nodos, relaciones y propiedades. Su documentación lo presenta justamente como una base donde las relaciones se

almacenan de forma explícita en lugar de quedar implícitas como en tablas o documentos. Por eso resulta especialmente potente cuando el problema principal es analizar vínculos complejos, recorridos, comunidades, dependencias o redes de interacción.

Redis ocupa otro lugar. Su documentación lo describe como un servidor de estructuras de datos que ayuda a resolver problemas que van desde caché hasta colas y procesamiento de eventos. El foco no está en representar documentos ricos ni en navegar relaciones profundas, sino en responder rápido, sostener estado compartido, aplicar reglas simples y trabajar muy bien con temporalidad y expiración.

Por eso, la pregunta correcta no es “¿cuál es mejor?”, sino “¿qué problema estoy resolviendo?”. Si necesitás rapidez extrema, estado transitorio, sesiones, contadores, expiración y decisiones inmediatas, Redis suele ser una gran elección. Si necesitás consultas documentales más ricas y estructuras flexibles por entidad, un modelo documental tiene más sentido. Si el valor está en las conexiones entre entidades, en recorridos o patrones relacionales complejos, un modelo grafo será más natural.

La síntesis esperada para el alumno sería esta: sí elegir Redis cuando el problema exige velocidad, baja latencia, estado momentáneo y control temporal; no elegirlo como solución principal cuando el corazón del problema son consultas complejas, análisis documental profundo o relaciones ricas entre entidades. En sistemas modernos, de hecho, lo más habitual no es que compita solo contra otros modelos, sino que conviva con ellos dentro de una arquitectura híbrida donde cada tecnología hace aquello para lo que fue mejor pensada.

Guía de instalación de Redis paso a paso para la clase

En esta materia vamos a instalar y ejecutar Redis en Windows usando Docker. Esta es la vía que hoy recomienda la documentación oficial de Redis para Windows: primero instalar Docker Desktop y luego correr Redis dentro de un contenedor. Redis, además, indica que la conexión con `redis-cli` puede hacerse desde el mismo contenedor si no está instalada localmente la herramienta.

Los enlaces oficiales que vas a necesitar son estos: la página de instalación de Docker Desktop para Windows, la guía oficial de Redis en Windows con Docker, la guía oficial de Redis en Docker y, de manera opcional, la guía oficial de Redis Insight en Docker. Podés abrir cada uno desde su cita.

Docker Desktop será el programa que permitirá ejecutar contenedores Linux dentro de Windows. Redis correrá dentro de uno de esos contenedores, escuchando normalmente en el puerto 6379. Este enfoque evita instalaciones distintas según cada máquina y deja a todos los alumnos trabajando con el mismo entorno. Redis, en su guía oficial para Windows, remite justamente a esta forma de trabajo.

Requisitos previos antes de empezar

Antes de instalar Docker Desktop, conviene revisar que la máquina cumpla las condiciones mínimas. La documentación de Docker para Windows indica, para el backend WSL 2, que se necesita WSL 2 versión 2.1.5 o posterior, una edición compatible de Windows 10 u 11 de 64 bits, al menos 8 GB de RAM y virtualización de hardware habilitada en BIOS/UEFI. También aclara que Docker Desktop no está soportado en Windows Server.

Si Docker ya estuvo instalado antes en la máquina, igual conviene abrirlo y comprobar que esté funcionando correctamente antes de seguir. En esta guía vamos a trabajar con contenedores Linux, que es justamente el modo que se usa para Redis en Windows.

Paso 1: instalar Docker Desktop

Entrá a la página oficial de instalación de Docker Desktop para Windows y descargá el instalador. <https://docs.docker.com/desktop/setup/install/windows-install/Descargar>. La propia guía de Docker indica que la descarga puede hacerse desde el botón principal de la página de instalación o desde las release notes.

Una vez descargado el archivo, hace doble clic sobre Docker Desktop Installer.exe. Docker indica que, por defecto, el programa se instala en C:\Program Files\Docker\Docker. Durante la instalación, si aparece la pantalla de configuración, asegurate de dejar seleccionada la opción **Use WSL 2 instead of Hyper-V** si esa es la alternativa que vas a usar.

Cuando termine la instalación, reiniciá Windows si el instalador lo solicita. Después abrí Docker Desktop desde el menú Inicio y esperá a que el motor quede activo. Visualmente, Docker Desktop debe mostrar que está corriendo y sin errores. Esto es importante porque Redis se ejecutará dentro de Docker y, si Docker no está levantado, ninguno de los comandos de los próximos pasos funcionará.

Paso 2: verificar que Docker está funcionando

Abrí PowerShell y ejecutá este comando:

```
docker --version
```

Este comando sirve para verificar que Docker quedó instalado y que el sistema reconoce el ejecutable.

Después ejecutá:

```
docker ps
```

Si Docker está funcionando, este comando no debería dar error. Puede mostrar una lista vacía de contenedores, y eso es normal si todavía no levantaste ninguno.

Si en este punto aparece un error relacionado con WSL, virtualización o arranque del motor, no sigas todavía con Redis. Primero hay que resolver Docker Desktop. Las causas más comunes, según la documentación oficial, son WSL 2 no habilitado, virtualización desactivada en BIOS/UEFI o una versión de Windows fuera de los requisitos.

Paso 3: descargar y ejecutar Redis

La guía oficial de Redis para Docker indica este formato de comando para arrancar Redis:

```
docker run -d --name redis -p 6379:6379 redis:<version>
```

Ese es el patrón oficial: crear un contenedor, ponerle nombre y publicar el puerto 6379.

Para la clase, te propongo usar este comando directamente en PowerShell:

```
docker run -d --name redis-clase -p 6379:6379 redis:latest
```

Qué hace este comando:

- docker run crea y ejecuta un contenedor nuevo.
- d lo deja corriendo en segundo plano.
- name redis-clase le asigna un nombre fácil de recordar.
- p 6379:6379 conecta el puerto 6379 del contenedor con el 6379 de tu máquina.
- redis:latest indica la imagen de Redis que Docker debe descargar y usar.

La primera vez puede tardar un poco porque Docker tiene que bajar la imagen desde el repositorio correspondiente. Cuando termina, devuelve un identificador largo del contenedor. Eso normalmente indica que Redis ya quedó creado y ejecutándose.

Paso 4: comprobar que Redis está corriendo

Una vez ejecutado el paso anterior, escribí:

```
docker ps
```

En la salida debería aparecer un contenedor llamado redis-clase y deberías ver el puerto 6379 publicado.

Si querés revisar más detalle, podés usar:

```
docker logs redis-clase
```

Este comando muestra el log del contenedor. Si Redis arrancó bien, verás mensajes relacionados con el inicio del servidor. Si el contenedor se cerró enseguida o hubo algún conflicto, este comando ayuda a detectarlo.

Paso 5: conectarte a Redis con redis-cli

La documentación oficial de Redis aclara que, si no tenés redis-cli instalado localmente, podés ejecutarlo desde el mismo contenedor con este formato:

```
docker exec -it redis redis-cli
```

Eso permite entrar al cliente interactivo sin instalar herramientas extra en Windows.

En nuestra instalación, como el contenedor se llama redis-clase, el comando queda así:

```
docker exec -it redis-clase redis-cli
```

Si todo está bien, vas a ver un prompt de Redis. A partir de ese momento ya estás dentro del cliente y podés probar los primeros comandos.

Paso 6: prueba inicial para confirmar que funciona

Dentro de redis-cli, ejecutá lo siguiente:

```
PING
SET prueba "hola redis"
GET prueba
EXISTS prueba
DEL prueba
EXISTS prueba
```

Qué debería pasar:

PING debería responder PONG.
SET debería responder OK.
GET prueba debería devolver "hola redis".
EXISTS prueba debería devolver 1 mientras la clave exista.
Después de DEL prueba, EXISTS prueba debería devolver 0.

Si esto ocurre, Redis ya está instalado y funcionando correctamente en tu máquina.

Paso 7: cerrar, detener y volver a iniciar Redis

Cuando terminás de trabajar en redis-cli, escribí:

```
QUIT
```

Eso cierra solo el cliente, no el servidor.

Si querés detener el contenedor Redis:

```
docker stop redis-clase
```

Si después querés volver a iniciarlo sin reinstalar nada:

```
docker start redis-clase
```

Si querés eliminar completamente el contenedor:

```
docker rm -f redis-clase
```

Estos comandos son útiles para administrar el entorno de práctica durante la cursada.

Paso opcional: instalar Redis Insight para trabajar con interfaz gráfica

Redis también ofrece Redis Insight como herramienta gráfica oficial. Su documentación en Docker muestra este comando para levantarla:

```
docker run -d --name redisinsight -p 5540:5540  
redis/redisinsight:latest
```

Y luego indica abrir el navegador en <http://localhost:5540>.

En PowerShell, ejecutá:

```
docker run -d --name redisinsight -p 5540:5540  
redis/redisinsight:latest
```

Después abrí el navegador y entrá a:

<http://localhost:5540>

Cuando Redis Insight abra, agregá una nueva conexión y usá estos datos:

```
Host: 127.0.0.1  
Port: 6379
```

Esto te permitirá ver claves, hashes, sets, TTL y demás estructuras de una forma mucho más visual, lo cual ayuda bastante en las primeras prácticas.

Problemas frecuentes y cómo revisarlos

Si `docker --version` funciona pero `docker ps` da error, probablemente Docker Desktop todavía no está arrancado del todo o hay un problema con WSL 2 o virtualización. Docker pide WSL 2 habilitado, virtualización activada en BIOS/UEFI y requisitos mínimos de hardware y versión de Windows compatibles.

Si al ejecutar Redis aparece un error de puerto ocupado, significa que el puerto 6379 ya está siendo usado por otro proceso. En ese caso podés cambiar el puerto externo, por ejemplo así:

```
docker run -d --name redis-clase -p 6380:6379 redis:latest
```

Y luego conectarte con el cliente local al puerto 6380. Si entrás con docker exec al contenedor, no hace falta preocuparse por ese cambio porque el cliente se ejecuta dentro del propio contenedor.

Si redis-cli no está instalado en Windows, eso no es un problema. Redis documenta explícitamente que puede usarse docker exec -it redis redis-cli para abrir el cliente desde el contenedor. En nuestra práctica, simplemente reemplazamos redis por redis-clase.

Si Redis Insight abre pero no conecta, comprobá primero que Redis siga corriendo con:

```
docker ps
```

y revisá que el contenedor redis-clase esté activo y que el puerto publicado sea el correcto.

Trabajo guiado –Redis, clave–valor, estado y temporalidad

A continuación tenés una práctica guiada para trabajar Redis paso a paso. La idea no es ejecutar comandos sueltos, sino entender cómo una base clave–valor puede resolver situaciones reales de control, monitoreo y automatización. Durante la actividad vas a crear datos, consultarlos, modificarlos, trabajar con contadores, usar conjuntos y aplicar expiración temporal.

El escenario de trabajo será el siguiente: una empresa ejecuta controles automáticos sobre procesos de auditoría. El sistema necesita registrar el estado de tareas, guardar datos breves de procesos, mantener listas de proveedores observados, contar intentos fallidos de acceso y activar alertas temporales.

Antes de comenzar, abrí redis-cli. Si querés empezar desde cero, podés limpiar la base actual. Hacelo solo si estás seguro de que no necesitás conservar nada de lo que ya cargaste.

```
FLUSHDB
```

Objetivo de la práctica: Comprender cómo se almacenan datos en Redis mediante claves; usar claves simples, hashes, sets y contadores; consultar y modificar información operativa; aplicar TTL para manejar datos temporales; interpretar a Redis como una herramienta útil para caché, estado y control automatizado.

Primera parte: crear datos iniciales

Primero, crear el estado de una tarea automática, el último resultado de un control y una validación rápida sobre un proveedor.

```
SET tarea:lote_001 "pendiente"
SET control:ultima_revision "sin_desvios"
SET validacion:proveedor_203 "aprobado"
```

Ahora, crear dos procesos de auditoría usando hashes. Cada proceso tendrá varios atributos.

```
HSET auditoria:proc_101 estado "en_ejecucion" fecha_inicio "2026-04-12" usuario
"alfonso" resultado "pendiente"
HSET auditoria:proc_102 estado "finalizado" fecha_inicio "2026-04-11" usuario "laura"
resultado "sin_desvios"
```

Después, crear tres conjuntos: usuarios bloqueados, proveedores observados y transacciones en revisión.

```
SADD usuarios:bloqueados "u102" "u330"
SADD proveedores:observados "203" "450" "778"
SADD transacciones:revision "trx_9001" "trx_9002" "trx_9010"
```

Ahora, cargar algunos contadores iniciales.

```
SET login:intentos:u102 3
SET login:intentos:u330 5
SET alertas:fraude 2
SET excepciones:lote_001 4
```

Finalmente, crear algunas claves temporales.

```
SET alerta:proveedor_203 "activa"
EXPIRE alerta:proveedor_203 3600
```

```
SET cache:reporte_mensual "generado"
EXPIRE cache:reporte_mensual 1800
```

```
SET login:bloqueado:u330 "1"
EXPIRE login:bloqueado:u330 600
```

Segunda parte: consultar lo cargado

Ahora vas a leer y verificar la información para comprobar que Redis almacenó todo correctamente.

Consultar claves simples:

GET tarea:lote_001
GET control:ultima_revision
GET validacion:proveedor_203

Consultar hashes:

HGET auditoria:proc_101 estado
HGET auditoria:proc_101 usuario
HGETALL auditoria:proc_101
HGETALL auditoria:proc_102

Consultar sets:

SMEMBERS usuarios:bloqueados
SMEMBERS proveedores:observados
SMEMBERS transacciones:revision

Verificar pertenencia puntual dentro de un set:

SISMEMBER usuarios:bloqueados "u102"
SISMEMBER proveedores:observados "203"
SISMEMBER transacciones:revision "trx_9999"

Consultar contadores:

GET login:intentos:u102
GET login:intentos:u330
GET alertas:fraude
GET excepciones:lote_001

Consultar TTL y existencia de claves:

TTL alerta:proveedor_203
TTL cache:reporte_mensual
TTL login:bloqueado:u330

EXISTS tarea:lote_001
EXISTS alerta:proveedor_203
EXISTS clave:inexistente

En este punto, deberías observar que Redis devuelve valores muy rápidamente, que los hashes agrupan atributos de una misma entidad, que los sets permiten verificar pertenencia y que las claves temporales muestran cuánto tiempo les queda de vida.

Tercera parte: modificar los datos

Ahora vas a simular cambios en el sistema.

Modificar claves simples:

```
SET tarea:lote_001 "en_ejecucion"  
SET control:ultima_revision "con_alerta"  
SET validacion:proveedor_203 "observado"
```

Modificar atributos dentro de un hash:

```
HSET auditoria:proc_101 estado "finalizado"  
HSET auditoria:proc_101 resultado "con_desvios"  
HSET auditoria:proc_101 fecha_fin "2026-04-12"  
HSET auditoria:proc_101 reintentos 1
```

Modificar sets, agregando y quitando elementos:

```
SADD usuarios:bloqueados "u450"  
SADD proveedores:observados "999"  
SADD transacciones:revision "trx_9025"
```

```
SREM usuarios:bloqueados "u330"  
SREM proveedores:observados "450"  
SREM transacciones:revision "trx_9002"
```

Modificar contadores:

```
INCR login:intentos:u102  
INCR alertas:fraude  
INCR excepciones:lote_001
```

```
DECR login:intentos:u330  
DECR alertas:fraude
```

```
INCRBY excepciones:lote_001 3
```

Modificar expiraciones:

```
EXPIRE alerta:proveedor_203 7200  
EXPIRE cache:reporte_mensual 3600  
PERSIST alerta:proveedor_203
```

Acá conviene prestar atención a dos ideas. La primera es que en Redis muchas modificaciones se hacen reescribiendo el valor o actualizando directamente el campo. La segunda es que el tiempo también forma parte del diseño: una clave puede extender su vigencia o dejar de ser temporal.

Cuarta parte: volver a consultar y analizar

Después de modificar, repetir algunas consultas para comparar el antes y el después.

GET tarea:lote_001
GET control:ultima_revision
GET validacion:proveedor_203

HGETALL auditoria:proc_101

SMEMBERS usuarios:bloqueados
SMEMBERS proveedores:observados
SMEMBERS transacciones:revision

GET login:intentos:u102
GET login:intentos:u330
GET alertas:fraude
GET excepciones:lote_001

TTL alerta:proveedor_203
TTL cache:reporte_mensual

La finalidad de esta parte no es solo técnica. También tenés que interpretar qué pasó en el sistema. Por ejemplo: la tarea ya no está pendiente, un proveedor pasó de aprobado a observado, un proceso cambió su resultado, ciertos elementos dejaron de pertenecer a un conjunto y algunos contadores aumentaron por eventos nuevos.

Quinta parte: eliminación de datos

Ahora vas a eliminar algunas claves para observar el comportamiento.

DEL tarea:lote_001
DEL control:ultima_revision
DEL auditoria:proc_102
DEL alerta:proveedor_203

Comprobá luego si siguen existiendo:

EXISTS tarea:lote_001
EXISTS control:ultima_revision
EXISTS auditoria:proc_102
EXISTS alerta:proveedor_203

Esta parte ayuda a reforzar la idea de ciclo de vida del dato. En Redis no todo está pensado para permanecer para siempre.

Anexo – Laboratorio introductorio de Redis

1. Propósito del laboratorio

Este anexo tiene como objetivo presentar una práctica básica y guiada sobre Redis, orientada a comprender cómo se almacenan, consultan y modifican datos mediante comandos simples. La intención no es memorizar comandos, sino interpretar qué tipo de problema resuelve cada estructura.

Redis trabaja como una base de datos clave–valor en memoria. Esto significa que cada dato se almacena asociado a una clave, y el acceso se realiza directamente a partir de esa clave. Sin embargo, Redis no se limita a guardar valores simples: también permite utilizar estructuras como hashes, sets, listas, sorted sets y contadores. Por eso se lo suele definir como un almacén de estructuras de datos en memoria.

En esta práctica se trabajará con los tipos más importantes para la clase:

- Strings o valores simples.
- Hashes.
- Sets.
- Listas.
- Sorted Sets.
- Contadores.
- TTL y expiración.
- Comandos de consulta del espacio de claves.
- Transacciones básicas.

2. Antes de comenzar

Todos los comandos se ejecutan desde redis-cli.

Si se está trabajando con Redis en Docker, primero se debe abrir la consola de Redis con: **docker exec -it redis-clase redis-cli**

Para verificar que Redis está funcionando:

PING

Resultado esperado:

PONG

Si se desea comenzar con una base limpia para la práctica:

FLUSHDB

Este comando elimina las claves de la base actual. Debe usarse solo en entornos de práctica.

3. Strings: clave–valor simple

El tipo más básico de Redis es el string. Permite guardar un valor simple asociado a una clave. Ese valor puede ser texto, número o incluso contenido serializado, como un JSON. En la práctica de laboratorio base se presenta SET como comando de inserción o actualización, y GET como comando de recuperación del valor asociado a una clave.

3.1 Crear y consultar valores

```
SET sistema:estado "activo"
```

```
GET sistema:estado
```

Resultado esperado:

```
"activo"
```

Otro ejemplo:

```
SET tarea:lote_001 "pendiente"
```

```
GET tarea:lote_001
```

Aquí la clave es tarea:lote_001 y el valor es "pendiente".

Redis no interpreta semánticamente la clave. Para Redis, tarea:lote_001 es solo un string. El significado lo aporta el diseño de la aplicación.

3.2 Sobrescribir un valor

```
SET tarea:lote_001 "en_ejecucion"
```

```
GET tarea:lote_001
```

Resultado esperado:

```
"en_ejecucion"
```

En Redis, SET crea la clave si no existe o sobrescribe su valor si ya existe.

3.3 Verificar existencia

```
EXISTS tarea:lote_001
```

Resultado esperado:

```
(integer) 1
```

Si la clave no existe:

```
EXISTS tarea:inexistente
```

Resultado esperado:

(integer) 0

3.4 Eliminar una clave

DEL tarea:lote_001

EXISTS tarea:lote_001

Resultado esperado:

(integer) 0

DEL elimina la clave y el valor asociado. Este comando es útil cuando un estado operativo ya no tiene sentido.

4. MSET y MGET: operaciones sobre múltiples claves

Redis permite crear o consultar varias claves en un único comando. El laboratorio de referencia presenta MSET y MGET como operaciones masivas para modificar u obtener valores de múltiples claves.

4.1 Cargar varias claves

MSET control:001 "ok" control:002 "pendiente" control:003 "error"

4.2 Consultarlas juntas

MGET control:001 control:002 control:003

Resultado esperado:

- 1) "ok"
- 2) "pendiente"
- 3) "error"

Este patrón es útil cuando la aplicación necesita consultar varios estados conocidos al mismo tiempo.

5. TYPE: saber qué tipo de dato guarda una clave

Redis es tipado por clave. Una clave puede contener un string, hash, set, lista, sorted set, etc. El comando TYPE permite saber qué tipo de estructura tiene asociada una clave. En el laboratorio base se muestra que TYPE devuelve el tipo almacenado o none si la clave no existe.

SET usuario:45:estado "activo"

TYPE usuario:45:estado

Resultado esperado:

string

Si la clave no existe:

TYPE usuario:999

Resultado esperado:

none

Este comando es importante porque no se deben mezclar comandos de distintos tipos. Si una clave fue creada como string, no se puede consultar como hash o set.

6. Contadores: INCR, INCRBY, DECR y DECRBY

Un contador en Redis es una clave cuyo valor es numérico y puede incrementarse o decrementarse de forma atómica. El material de laboratorio explica que INCR parsea el valor como entero, lo incrementa y guarda el resultado; además, destaca que esto evita condiciones de carrera cuando múltiples clientes incrementan la misma clave concurrentemente.

6.1 Crear e incrementar un contador

SET login:intentos:usuario_45 0

INCR login:intentos:usuario_45

INCR login:intentos:usuario_45

GET login:intentos:usuario_45

Resultado esperado:

"2"

6.2 Incrementar por una cantidad

INCRBY login:intentos:usuario_45 3

GET login:intentos:usuario_45

Resultado esperado:

"5"

6.3 Decrementar

DECR login:intentos:usuario_45

GET login:intentos:usuario_45

Resultado esperado:

"4"

6.4 Caso aplicado

Supongamos que un sistema quiere bloquear temporalmente a un usuario si supera cinco intentos fallidos. El contador registra el comportamiento:

INCR login:intentos:usuario_45

Luego, la aplicación evalúa el valor. Si supera el límite, puede crear una clave de bloqueo:

SET login:bloqueado:usuario_45 "1" EX 600

Esto representa un bloqueo por 10 minutos.

7. TTL y expiración automática

Una de las capacidades más importantes de Redis es asignar vida útil a una clave. El comando EXPIRE permite definir cuántos segundos vivirá una clave. En el laboratorio base se menciona que SET puede recibir opciones como EX para expiración en segundos, PX para milisegundos, NX para crear solo si la clave no existe y XX para operar solo si la clave ya existe.

7.1 Crear una clave con expiración

SET alerta:proveedor_203 "activa" EX 60

La clave existirá durante 60 segundos.

7.2 Consultar tiempo restante

TTL alerta:proveedor_203

Redis devuelve los segundos restantes.

7.3 Crear primero y luego asignar expiración

SET cache:reporte_ventas "resultado_generado"

EXPIRE cache:reporte_ventas 120

TTL cache:reporte_ventas

7.4 Eliminar el TTL y dejar la clave permanente

PERSIST cache:reporte_ventas

TTL cache:reporte_ventas

Si TTL devuelve -1, significa que la clave existe pero ya no tiene expiración.

7.5 Caso aplicado

SET login:bloqueado:usuario_45 "1" EX 600

Interpretación:

- La clave indica que el usuario está bloqueado.
- El valor "1" representa la condición de bloqueo.
- EX 600 indica que el bloqueo dura 600 segundos.

- Cuando el tiempo termina, Redis elimina automáticamente la clave.

La aplicación puede consultar:

GET login:bloqueado:usuario_45

Si devuelve "1", rechaza el acceso. Si devuelve nil, el usuario ya no está bloqueado.

8. Hashes: agrupar campos dentro de una clave

Los hashes permiten guardar campos y valores dentro de una única clave. El laboratorio base los presenta como estructuras formadas por pares campo–valor que pueden consultarse con HGET, HGETALL y modificarse con comandos como HSET o HINCRBY.

8.1 Crear un hash

HSET auditoria:proc_101 estado "en_ejecucion" usuario "aflores" fecha "2026-04-12" resultado "pendiente"

Aquí la clave es:

auditoria:proc_101

Y dentro del hash se guardan campos:

estado
usuario
fecha
resultado

8.2 Consultar un campo

HGET auditoria:proc_101 estado

Resultado esperado:

"en_ejecucion"

8.3 Consultar todo el hash

HGETALL auditoria:proc_101

Resultado esperado aproximado:

- 1) "estado"
- 2) "en_ejecucion"
- 3) "usuario"
- 4) "aflores"
- 5) "fecha"
- 6) "2026-04-12"

7) "resultado"

8) "pendiente"

8.4 Actualizar un campo

HSET auditoria:proc_101 estado "finalizado"

HSET auditoria:proc_101 resultado "con_desvios"

8.5 Incrementar un campo numérico dentro de un hash

HSET auditoria:proc_101 score 80

HINCRBY auditoria:proc_101 score 5

HGET auditoria:proc_101 score

Resultado esperado:

"85"

8.6 Cuándo usar hashes

Conviene usar hashes cuando varios atributos pertenecen a una misma entidad lógica.

Ejemplo:

auditoria:proc_101

puede contener:

- estado
- usuario
- fecha
- resultado
- score
- reintentos

Esto evita tener muchas claves sueltas como:

auditoria:proc_101:estado

auditoria:proc_101:usuario

auditoria:proc_101:fecha

La regla práctica es: si varios datos forman parte del mismo objeto operativo, un hash suele ser una buena opción.

9. Sets: conjuntos sin repetidos

Los sets son conjuntos de valores únicos, sin orden. En el laboratorio base se indica que los sets permiten agregar y eliminar elementos, consultar pertenencia, obtener cardinalidad y realizar operaciones como unión, intersección y diferencia.

9.1 Crear un set

SADD usuarios:bloqueados "u102" "u330" "u450"

9.2 Consultar todos los elementos

SMEMBERS usuarios:bloqueados

Resultado esperado:

- 1) "u102"
- 2) "u330"
- 3) "u450"

El orden puede variar porque un set no mantiene orden.

9.3 Verificar pertenencia

SISMEMBER usuarios:bloqueados "u102"

Resultado esperado:

(integer) 1

Si el usuario no pertenece al set:

SISMEMBER usuarios:bloqueados "u999"

Resultado esperado:

(integer) 0

SISMEMBER no explora el set como una búsqueda tradicional. Verifica pertenencia de forma directa.

9.4 Eliminar un elemento

SREM usuarios:bloqueados "u330"

SMEMBERS usuarios:bloqueados

9.5 Cardinalidad

SCARD usuarios:bloqueados

Devuelve la cantidad de elementos del set.

9.6 Caso aplicado

SADD proveedores:observados "203" "450" "778"

SISMEMBER proveedores:observados "203"

La aplicación puede interpretar 1 como “proveedor observado” y 0 como “proveedor no observado”.

10. Operaciones entre sets

Los sets también permiten operaciones de teoría de conjuntos. El laboratorio de referencia incluye unión, intersección y diferencia entre conjuntos mediante comandos como SUNION, SINTER y SDIFF.

10.1 Crear dos conjuntos

SADD proveedores:observados "203" "450" "778"

SADD proveedores:con_pagos_recientes "203" "999" "450"

10.2 Intersección

SINTER proveedores:observados proveedores:con_pagos_recientes

Resultado esperado:

1) "203"

2) "450"

Interpretación: proveedores observados que además tienen pagos recientes.

10.3 Unión

SUNION proveedores:observados proveedores:con_pagos_recientes

Devuelve todos los proveedores de ambos conjuntos, sin duplicados.

10.4 Diferencia

SDIFF proveedores:observados proveedores:con_pagos_recientes

Devuelve los proveedores observados que no están en el conjunto de proveedores con pagos recientes.

11. Listas: secuencia ordenada de valores

Las listas guardan valores ordenados. Pueden recibir elementos al inicio o al final. El laboratorio base señala que Redis implementa listas como listas enlazadas, por lo que agregar al principio o al final es eficiente, aunque el acceso por índice intermedio no es tan performante como en un array.

11.1 Crear una lista

LPUSH cola:tareas "validar_proveedor"

LPUSH cola:tareas "revisar_pago"

RPUSH cola:tareas "enviar_alerta"

11.2 Consultar la lista

LRange cola:tareas 0 -1

Resultado posible:

- 1) "revisar_pago"
- 2) "validar_proveedor"
- 3) "enviar_alerta"

11.3 Extraer elementos

LPOP cola:tareas

RPOP cola:tareas

LPOP elimina y devuelve el primer elemento.

RPOP elimina y devuelve el último elemento.

11.4 Longitud de la lista

LLEN cola:tareas

11.5 Cuándo usar listas

Las listas sirven para:

- colas simples de tareas;
- últimos eventos;
- comunicación productor–consumidor;
- historial breve y ordenado.

Ejemplo aplicado:

RPUSH cola:auditoria "proc_101"

RPUSH cola:auditoria "proc_102"

LPOP cola:auditoria

Un proceso puede insertar tareas y otro consumirlas.

12. Sorted Sets: conjuntos ordenados por score

Un sorted set es similar a un set porque no repite elementos, pero agrega un score numérico que permite ordenar. El laboratorio base explica que cada elemento tiene asociado un score, y que Redis mantiene los valores ordenados.

12.1 Crear un ranking

ZADD ranking:riesgo 90 "proveedor_203"

ZADD ranking:riesgo 75 "proveedor_450"

ZADD ranking:riesgo 95 "proveedor_778"

12.2 Consultar en orden ascendente

ZRANGE ranking:riesgo 0 -1 WITHSCORES

12.3 Consultar en orden descendente

ZREVRANGE ranking:riesgo 0 -1 WITHSCORES

12.4 Obtener los proveedores de mayor riesgo

ZREVRANGE ranking:riesgo 0 1 WITHSCORES

Resultado esperado: los dos proveedores con mayor score.

12.5 Consultar por rango de score

ZRANGEBYSCORE ranking:riesgo 80 100 WITHSCORES

Devuelve los elementos cuyo score esté entre 80 y 100.

12.6 Cuándo usar sorted sets

Son útiles para:

- rankings;
- scoring;
- prioridades;
- top N;
- ordenamiento por riesgo;
- eventos con puntaje.

En auditoría, se pueden usar para ordenar proveedores, transacciones o procesos según nivel de riesgo.

13. SCAN: recorrer claves de forma incremental

En Redis no es buena práctica usar KEYS * en producción, porque puede bloquear o afectar el rendimiento si hay muchas claves. El laboratorio base presenta la familia SCAN, SSCAN, HSCAN y ZSCAN para iterar incrementalmente sobre claves, sets, hashes y sorted sets.

13.1 Uso básico

SCAN 0

Redis devuelve un cursor y un conjunto parcial de claves.

La iteración continúa usando el cursor devuelto:

SCAN <cursor>

Cuando el cursor vuelve a 0, terminó la iteración.

13.2 Buscar con patrón

SCAN 0 MATCH auditoria:*

Este comando ayuda a explorar claves por patrón, pero debe usarse con criterio. En diseños bien pensados, la aplicación debería conocer las claves que necesita consultar.

14. Transacciones básicas: MULTI y EXEC

Redis permite agrupar comandos para que se ejecuten secuencialmente. El laboratorio base explica que MULTI inicia una transacción, los comandos se encolan, y EXEC dispara la ejecución del bloque. También aclara que Redis no trabaja con rollback como una base relacional tradicional, porque su simplicidad y velocidad se apoyan en otro modelo de operación.

14.1 Ejemplo simple

Supongamos que queremos registrar que un usuario dio "like" a una publicación y, al mismo tiempo, incrementar el contador de likes.

```
MULTI
SADD post:100:likes:users "usuario_45"
INCR post:100:likes:counter
EXEC
```

Interpretación:

- SADD agrega el usuario al set de usuarios que dieron like.
- INCR incrementa el contador.
- EXEC ejecuta ambos comandos en bloque.

14.2 Cuándo usarlas

Las transacciones básicas pueden servir cuando varias operaciones deben quedar agrupadas lógicamente.

Ejemplo:

- agregar usuario a un conjunto;
- incrementar contador;

- marcar estado;
- registrar una alerta.

No deben confundirse con transacciones complejas de SQL. Redis no está pensado para reemplazar transacciones relacionales con múltiples tablas, restricciones y rollback.

15. Buenas prácticas

15.1 Diseñar bien las claves

Una buena clave debe ser clara y semántica.

Ejemplo recomendable:

login:intentos:usuario_45

Ejemplo pobre:

x45flag

La clave debería permitir entender rápidamente:

- dominio o contexto;
- entidad;
- identificador;
- atributo o propósito operativo.

Ejemplo:

auditoria:proc_101:estado

15.2 No mezclar tipos para la misma clave

Si una clave fue creada como string:

SET usuario:45 "activo"

no debe tratarse luego como hash:

HGET usuario:45 estado

Esto generará error de tipo.

15.3 Elegir la estructura según el acceso

No se elige Redis por el comando, sino por la pregunta que se quiere responder.

Necesidad	Tipo sugerido	Comandos
Guardar un estado simple	String	SET, GET, DEL
Agrupar atributos de una entidad	Hash	HSET, HGET, HGETALL
Validar pertenencia	Set	SADD, SISMEMBER, SMEMBERS
Acumular eventos	String contador	INCR, DECR, INCRBY
Manejar cola simple	List	LPUSH, RPUSH, LPOP, RPOP
Ranking o score	Sorted Set	ZADD, ZRANGE, ZREVRANGE
Dato temporal	Cualquier clave con TTL	EXPIRE, TTL, SET ... EX

15.4 Usar TTL cuando el dato sea temporal

Ejemplos de datos temporales:

login:bloqueado:usuario_45
 session:usuario_45
 cache:reporte:ventas
 alerta:proveedor_203

En esos casos conviene definir expiración:

SET login:bloqueado:usuario_45 "1" EX 600

15.5 No usar Redis como repositorio de archivos grandes

Redis puede guardar strings binarios, pero no es la mejor opción para videos, PDFs pesados o imágenes grandes. Para eso conviene usar almacenamiento de objetos, como S3 o un sistema equivalente, y guardar en Redis solo referencias, estado o metadatos.

Ejemplo:

HSET documento:123 url "s3://bucket/doc123.pdf" estado "procesado" usuario "u45"